10.2 安全加固与权限管理

> **生成模型**：gpt-5.4 (openai/gpt-5.4)
> **Token 消耗**：输入 ~8k tokens，输出 ~2.6k tokens（估算）

前面几章我们一直在教你怎么把 OpenClaw 配起来、跑起来、接到更多渠道上。到了这一步，另外一个问题就必须正面回答了：**既然它能连聊天软件、能看文件、能跑工具，那到底怎么防止它帮错人、做错事、越权做事？**

这个问题不是“企业用户才需要担心”的大题目。你哪怕只是把 OpenClaw 接到 Telegram、Discord，或者开了一个能远程访问的 Gateway，安全就已经不是可选项了。因为对 Agent 来说，别人发来的一条自然语言消息，本身就可能是攻击入口。

所以这节不聊安全模型源码，也不讲很学院派的术语。我们就站在使用者角度，把最该锁的几道门说清楚。

10.2.1 第一原则：把入站消息都当成不可信

你可以把 OpenClaw 想成一位很能干、但有点过于热心的助理。它的问题不是“不会做事”，而是有时候太愿意照做。只要有人对它说得像模像样，它就可能认真执行。

所以安全的起点不是“相信它会自己分辨”，而是**默认外部输入不可信**。这包括：

- 陌生人的私聊。
- 群里的 @ 消息。
- 邮件钩子收到的正文。
- Webhook 推过来的数据。
- 它自己从网页抓下来的内容。

这个原则一旦立住，很多配置决策就顺了：谁能直接跟主 Agent 说话，谁只能进沙箱，谁根本不该拿到某些工具权限。

10.2.2 DM 配对：别让任何人都能直接私聊拿权限

如果你只记一个安全建议，那就是这个：**DM 一定优先用 pairing，不要图省事直接 open。**

所谓 DM pairing，本质上就是“先配对，再信任”。对方第一次私聊你家里的 OpenClaw 时，系统不会立刻把他当成熟人，而是要求完成一次配对确认。这样做很像蓝牙耳机第一次连手机时的配对码：多一步，但这一步能挡掉大量意外访问。

为什么它重要？因为 DM 在用户感觉上很“私人”，但从系统角度看，它恰恰是最容易让人放松警惕的入口。群聊里你至少还知道旁边有人看着，DM 一旦放开，陌生人就等于直接对着 Agent 耳边说话。

我的建议很简单：

- 个人自用：保留 pairing。
- 家人或小团队共用：仍然保留 pairing，再配 allowlist。
- 对公网开放：如果没有非常明确的隔离方案，就别把 DM 开成 open。

10.2.3 allowlist：谁能来，先定名单

配对解决的是“第一次能不能进来”，allowlist 解决的是“长期谁算可信对象”。

你可以把 allowlist 理解成白名单。上了名单的人或会话，系统才按你定义的规则处理；没上名单的，哪怕消息真的送到了，也不应该让它进入高权限路径。

这里很多人会犯一个很常见的误区：以为白名单只是“方便过滤打扰”。其实它更重要的作用是**收缩攻击面**。因为名单越小，可信边界越清楚，系统越不容易被奇怪输入绕进去。

如果是生产用途，建议你按最小范围来配：

- 能精确到账号，就别只按渠道放开。
- 能精确到单个群，就别整个渠道都开。
- 能只给一个 agent 用，就别设成全局默认。

说白了，不要写那种“先全开，后面再收”的配置。现实里“后面”往往就再也没做。

10.2.4 沙箱模式：不信任的会话，默认丢进隔离区

OpenClaw 一个很实用的安全思路是：**主会话和非主会话分开对待**。主会话通常是你自己在本机、CLI、受控环境里发起的；非主会话则可能来自聊天渠道、Webhook、定时任务，风险高很多。

这时候沙箱模式就很关键了。一个常见做法是给非主会话启用 sandbox：

```
{
  agents: {
    defaults: {
      sandbox: {
        mode: "non-main",
        workspaceAccess: "rw"
      }
    }
  }
}
```

这段配置背后的用户视角其实很好懂：我自己的主操作环境可以相对自由一点，但所有“不一定可信”的入口，默认先关进一个边界更明确的运行区。这样就算对方真的诱导 Agent 调工具、读文件、跑命令，伤害面也会小很多。

如果你的 OpenClaw 会接触公网、群聊或者自动化事件流，沙箱最好不要省。尤其是“看起来只是玩玩”的测试机器人，最容易在心态上放松，结果一路跑到线上。

10.2.5 工具策略：哪些工具能用，哪些绝对别给

安全不是只看“谁能进来”，还要看“进来之后能做什么”。这就轮到 tool policy 了，也就是工具的 allow / deny 列表。

举个很直白的例子：一个只负责整理文档的 agent，根本没必要拥有浏览器控制、节点控制、系统级执行这些能力。你给多了，不是更灵活，而是更危险。

一个典型思路像这样：

```
{
  agents: {
    list: [
      {
        id: "docs",
        tools: {
          allow: ["read", "write"],
          deny: ["bash", "browser", "nodes"]
        }
      }
    ]
  }
}
```

看着很朴素，但这是很典型的最小权限原则：够用就行，不多给。你甚至可以反过来理解，**真正专业的配置不是“什么都能做”，而是“每个 agent 只做它该做的事”。**

10.2.6 SOUL 安全：人格文件也不是绝对安全区

很多人第一次看到 `SOUL.md` ，会把它当成“随便写点风格设定”的地方。这没错，但只对了一半。因为 `SOUL.md` 会影响 Agent 的整体行为，如果里面混入恶意指令、越权偏好或者危险暗示，后果不会只是“说话口气变怪”，而可能是把系统往错误方向推。

所以你要把 SOUL 文件当成高信任配置来管理：

- 别从不明来源直接复制整份 `SOUL.md` 来用。
- 团队协作时，把它纳入版本管理和评审。
- 不要在里面写“无条件执行”“忽略安全限制”这种东西。
- 如果是公开模板，先当第三方代码看，再决定要不要引入。

一句话：**SOUL 可以塑造个性，但不能越过安全边界。**

10.2.7 生产使用时，到底该锁哪些地方

如果你准备把 OpenClaw 当成长期在线服务，而不是桌面小玩具，下面这张清单建议你逐条过一遍：

- DM 保持 pairing，不要无脑 open。
- 渠道入口加 allowlist，按账号、群组、agent 精确收口。
- 非主会话启用 sandbox，别让外部输入直接落宿主机。
- 给不同 agent 单独配工具权限，尤其限制 `bash` 、 `browser` 、 `nodes` 这类高风险工具。

- `SOUL.md` 、 `AGENTS.md` 、配置文件纳入版本控制，避免手工漂移。
- 定期跑 `openclaw doctor` 和 `openclaw security audit --deep` 。
- 把 API Key、密钥、令牌留在环境变量或专门密钥管理里，不写进记忆和提示文件。

如果只能做三件事，我会优先做：DM pairing、non-main sandbox、工具最小权限。这三样做好，已经能挡掉相当大一批真实风险。

本节小结

OpenClaw 的安全，不是靠某一个“高级功能”一招制敌，而是靠一层一层把边界收紧。先把所有外部输入都当成不可信，再用 DM pairing 控入口、用 allowlist 缩范围、用 sandbox 做隔离、用 tool policy 限权限，最后把 `SOUL.md` 这类高信任文件也纳入审慎管理。做到这一步，你的 OpenClaw 才更像一套能长期在线运行的系统，而不是一台碰运气的自动执行器。